Scalable Self-Play Reinforcement Learning via Hierarchical Spatial Transfer and Curriculum Training: A Case Study on Dots-and-Boxes

Nishant Mohan

April 2026

Abstract

Self-play reinforcement learning, exemplified by the AlphaZero paradigm, has achieved superhuman performance in classical board games such as Go, Chess, and Shogi. However, applying these methods to combinatorial games with distinct structural properties—delayed rewards, extra-turn mechanics, and rapidly growing state spaces—remains underexplored. We address this gap by developing a self-play system for Dots-and-Boxes, a deterministic two-player game whose state-space complexity ranges from 10^5 (2×2) to 10^{25} (5×5), and whose gameplay demands tight integration of local tactics with global strategic reasoning. Our work introduces three methodological contributions: (i) a *hierarchical patch-based network* (PatchNet) that transfers locally learned tactics from small boards to larger ones via frozen patch-level feature extraction and a trainable global aggregator, reducing trainable parameters by approximately 60%; (ii) a *phased training curriculum* that co-adapts search budget, learning rate, and exploration temperature across four stages to progressively shift the agent from exploration to exploitation; and (iii) a *board-size-invariant value target* ($z = \tanh(0.5\,\Delta)$) that preserves gradient magnitude across board scales, addressing the signal-crushing problem inherent in score-normalized targets. Experimental evaluation on 2×2 and 3×3 boards shows that the agent achieves a 100% win-rate against greedy baselines and up to 99% against depth-limited alpha–beta search, while requiring only hundreds of milliseconds per move compared to minutes for traditional search at equivalent strategic depth.

1 Introduction

Dots-and-Boxes is a deterministic, perfect-information, two-player, zero-sum pencil-and-paper game. Despite its simple rules—players alternate drawing edges on a grid; completing the fourth edge of a box yields a point and an extra turn—the game exhibits deep strategic complexity. The "chain rule" and parity arguments that govern expert play [4] make it a challenging testbed for learning-based game agents that must integrate local tactics (capturing free boxes) with global strategy (managing chain lengths and sacrifices).

The state-space complexity of Dots-and-Boxes grows rapidly: a 3×3 board already admits $\sim10^{10}$ states (Section 3.3), comparable to Connect Four ($\sim 10^{13}$), while a 5×5 board reaches $\sim 10^{25}$—between Chess ($\sim10^{47}$) and Checkers ($\sim10^{21}$) in scale [3]. Traditional alpha–beta search becomes infeasible beyond small boards and modest depths: even with move ordering and transposition tables, a depth-6 alpha–beta search on a 5×5 board requires ~7 minutes per move. This motivates learning-based approaches that amortize computation into a neural network and deploy it within a focused tree search.

We follow the AlphaZero paradigm [13, 12]: a policy–value network guides MCTS during self-play; training targets are derived from search visit counts (policy) and game outcomes (value). Our contributions are:

1. **PatchNet**: A hierarchical architecture that extracts overlapping patches from large boards, evaluates them with a frozen pre-trained small-board model, and trains only a lightweight global aggregator (Section 4.3).
2. **Phased curriculum**: A four-stage training schedule that adapts MCTS budget, learning rate, and temperature across training (Section 4.8.3).
3. **Board-size-invariant value target**: A scaled-tanh target $z = \tanh(0.5\,\Delta)$ that provides consistent gradient magnitude across board sizes (Section 4.8.2).
4. **Systems design**: An efficient training pipeline featuring asynchronous MCTS with pending-node scheduling, batched neural inference, dihedral symmetry augmentation, and DAG-based transposition reuse (Sections 4.4–4.8).

2 Background and Related Work

AlphaZero and self-play RL. AlphaGo Zero [13] demonstrated that a single neural network trained entirely from self-play—without human data—could surpass human-level performance in Go. The subsequent AlphaZero [12] generalised the approach to Chess and Shogi, showing that the combination of a policy–value network with MCTS and the PUCT selection rule [9] is a powerful domain-independent framework. MuZero [11] extended this further by learning a latent dynamics model, eliminating the need for a known game simulator during planning. Our work applies the AlphaZero paradigm (known rules, no learned dynamics) to Dots-and-Boxes, a game whose combinatorial structure differs markedly from Go and Chess.

MCTS foundations. Monte Carlo Tree Search was established by Coulom [5] and formalised with regret bounds by the UCT algorithm of Kocsis and Szepesvári [7]. Modern MCTS implementations use neural priors (PUCT) and Dirichlet exploration noise at the root. Our implementation additionally supports asynchronous leaf evaluation and DAG-based transposition reuse.

Dots-and-Boxes. The mathematical theory of Dots-and-Boxes, including chains, loops, and the Nimstring connection, was developed by Berlekamp [4]. Barker and Korf [3] solved 4×5 Dots-and-Boxes optimally using enhanced alpha–beta search with transposition tables and move ordering, establishing computational limits for exact methods. To our knowledge, no prior work applies deep RL self-play to Dots-and-Boxes at the scale presented here.

Replay buffers and experience replay. Experience replay [8] decorrelates training data by sampling from a buffer of past transitions. We use a capacity-limited circular buffer with adaptive growth across training phases.

Symmetry in game AI. Exploiting board symmetries for data augmentation is standard practice in AlphaZero-style systems. For Go, the dihedral group D_4 provides 8-fold augmentation on square boards [13]. We extend this to Dots-and-Boxes, handling the additional complexity of separate horizontal and vertical edge sets under rotation.

Transfer learning in games. Curriculum and transfer approaches have been explored in multi-agent settings [2]. Our PatchNet architecture (Section 4.3) is a novel form of spatial transfer: tactical knowledge learned on small boards is reused as a frozen feature extractor for larger boards.

3 Dots-and-Boxes Formalisation

3.1 Board Geometry

A board with $r \times c$ boxes has $(r{+}1)c$ horizontal edges and $r(c{+}1)$ vertical edges, for an action space of size

$$|\mathcal{A}| = (r{+}1)c + r(c{+}1) = 2rc + r + c. \tag{1}$$

Each action draws one currently undrawn edge. Completing the fourth edge of a box awards one point and grants an extra turn; otherwise the turn alternates.

3.2 Bitboard State Representation

The game state is encoded using four bitsets and two scalar scores:

- **Horizontal edges** $\mathbf{h} \in \{0,1\}^{(r+1)c}$: bit i is set iff horizontal edge i has been drawn.
- **Vertical edges** $\mathbf{v} \in \{0,1\}^{r(c+1)}$: analogous for vertical edges.
- **Box ownership** $\mathbf{b}_1, \mathbf{b}_2 \in \{0,1\}^{rc}$: bit j is set iff box j is owned by player 1 or 2.
- **Scores** $s_1, s_2 \in \mathbb{Z}_{\geq 0}$: number of captured boxes per player.
- **Current player** $p \in \{1,2\}$.

This representation enables $O(1)$ legality checks and incremental updates via bitwise operations.

3.3 State-Space Complexity

Let $E = 2rc + r + c$ be the number of edges and $B = rc$ the number of boxes. An upper bound on the number of distinct game states is

$$|\mathcal{S}| \leq 2^E \cdot 2^B \cdot 2 = 2^{E+B+1}, \tag{2}$$

where the 2^E factor covers edge patterns, 2^B covers ownership assignments for completed boxes[1], and the final factor of 2 accounts for the side to move.

Table 1 shows concrete magnitudes. The 5×5 state space ($\sim 10^{25}$) lies between Checkers ($\sim 5 \times 10^{20}$) [10] and Chess ($\sim 10^{47}$) [12] in scale, motivating the use of function approximation and focused tree search rather than exhaustive computation.

[1] Each completed box has exactly two possible owners (the player who drew the fourth edge), so 2^B is a tighter factor than the naïve 3^B (unclaimed/P1/P2).

Table 1: State-space upper bounds and action-space sizes for representative board sizes.

Board	E	B	$\lvert\mathcal{A}\rvert$	$\lvert\mathcal{S}\rvert \leq$
2×2	12	4	12	$2^{17} \approx 1.3 \times 10^5$
3×3	24	9	24	$2^{34} \approx 1.7 \times 10^{10}$
4×3	31	12	31	$2^{44} \approx 1.8 \times 10^{13}$
5×5	60	25	60	$2^{86} \approx 7.7 \times 10^{25}$
7×8	127	56	127	$2^{184} \approx 2.4 \times 10^{55}$

4 Method

4.1 System Architecture

Figure 1 summarises the training pipeline. Multiple self-play workers run MCTS in parallel, submitting leaf-evaluation requests to a centralised batched inference server. Completed games produce training tuples that are pushed to a replay buffer. A trainer thread samples minibatches, computes the combined loss, and updates the network via Adam [6]. The updated model is checkpointed and deployed for the next self-play iteration.

4.2 Policy–Value Network (AlphaZeroBitNet)

The base model is a fully-connected residual network with a shared trunk and two output heads.

Input features. The state is encoded as a flat feature vector composed of:

1. **Horizontal edge bits** $\in \{0,1\}^{(r+1)c}$
2. **Vertical edge bits** $\in \{0,1\}^{r(c+1)}$
3. **Box ownership** $\in \{-1,0,+1\}^{rc}$: $+1$ if owned by the current player, -1 if by the opponent, 0 if unclaimed
4. **Edge-count one-hot** $\in \{0,1\}^{rc\times 5}$: for each box, a 5-channel one-hot encoding of the number of drawn surrounding edges (0–4). The count-3 feature directly signals "free box" (one move from capture), while count-2 signals a half-open chain
5. **Game-phase scalar**: boxes_filled$/B \in [0,1]$
6. **Score-advantage scalar**: $(s_{\text{me}} - s_{\text{opp}})/B$

Yielding a total input size of $(r+1)c + r(c+1) + rc + 5rc + 2 = 2rc + r + c + 6rc + 2$.

Architecture.

- **Input projection:** Linear$(d_{\text{in}}, H) \to$ LayerNorm $\to$ ReLU.
- **Residual tower:** N_B fully-connected residual blocks. Each block applies Linear $\to$ LayerNorm $\to$ ReLU $\to$ Dropout(p) $\to$ Linear $\to$ LayerNorm, then adds a skip connection and applies ReLU. LayerNorm [1] is used instead of BatchNorm since self-play minibatches can be small.

Training Pipeline Architecture

W **Self-Play Workers** (parallel threads)

$\downarrow$

Each worker: Environment $\xrightarrow{\text{MCTS}}$ action

$\hookrightarrow$ leaf nodes $\xrightarrow{\text{submit}}$ InferenceServer

$\hookrightarrow$ receive $(\mathbf{p}, v)$, expand, backup

$\hookrightarrow$ root visit counts $\Rightarrow \pi$, play action

$\hookrightarrow$ on terminal: push (x, π, mask, z) to ReplayBuffer

---

InferenceServer (dedicated thread)

accumulate batch ($\leq B_{\max}$ states, 200 μs window)

$\rightarrow$ single forward pass $\rightarrow$ softmax $\rightarrow$ dispatch $(\mathbf{p}, v)$

Trainer (main thread)

sample minibatches from ReplayBuffer

$\rightarrow$ compute $\mathcal{L} = \mathcal{L}_\pi + 1.5\,\mathcal{L}_v$

$\rightarrow$ gradient clipping ($\|\nabla\|_2 \leq 1.0$)

$\rightarrow$ Adam step $\rightarrow$ checkpoint θ

Figure 1: End-to-end architecture of the training pipeline. Self-play workers, the inference server, and the trainer operate concurrently.

- **Policy head:** Linear$(H, H/2) \rightarrow$ LayerNorm $\rightarrow$ ReLU $\rightarrow$ Linear$(H/2, |\mathcal{A}|)$, producing logits over all edges.
- **Value head (deep MLP):** Linear$(H, H/2) \rightarrow$ LayerNorm $\rightarrow$ ReLU $\rightarrow$ Linear$(H/2, H/4) \rightarrow$ LayerNorm $\rightarrow$ ReLU $\rightarrow$ Linear$(H/4, 1) \rightarrow$ tanh.

The value head is deliberately deeper than the single-layer head used in standard AlphaZero [12]: Dots-and-Boxes requires aggregating global signals such as total score advantage, game phase, and chain parity, which benefit from additional nonlinear capacity.

Illegal move masking. During both training and inference, an additive mask is applied before the softmax:

$$\hat{\mathbf{p}} = \text{softmax}\Big(\mathbf{l} + (\mathbf{m} - \mathbf{1}) \cdot 10^9\Big), \tag{3}$$

where $\mathbf{l}$ are the raw logits and $\mathbf{m} \in \{0, 1\}^{|\mathcal{A}|}$ is the legal-action mask. The large negative bias drives illegal-action probabilities to zero without requiring architecture changes. The replay buffer stores this mask alongside each training sample to ensure consistent masking during gradient computation.

4.3 Hierarchical Patch-Based Network (PatchNet)

Training a monolithic network on larger boards (e.g., 5×5, 7×8) is expensive because both the network capacity and MCTS budget must scale with the state-space size. We introduce PatchNet, a hierarchical architecture that transfers locally learned tactics from a small pre-trained model to larger boards.

Core idea. Local tactical patterns in Dots-and-Boxes—such as "avoid giving away a free box" (count-3 avoidance) or "capture a chain"—are *spatially local*: they depend only on a small neighbourhood of the board. PatchNet exploits this by:

1. Extracting all overlapping $k \times k$ patches from the $r \times c$ board (yielding $(r-k+1)(c-k+1)$ patches).
2. Running a *frozen* pre-trained $k \times k$ AlphaZeroBitNet on each patch, producing local policy logits and a local value signal.
3. Scattering local policy logits to global action indices (averaging where patches overlap).
4. Concatenating the global feature representation, the scattered policy map, and the per-patch values into a fusion vector.
5. Processing the fusion vector through a trainable global aggregator (residual tower + policy/value heads) to produce the final output.

Architecture details. For a 5×5 board with 3×3 patches, there are $3 \times 3 = 9$ overlapping patches. Each patch produces 24 action logits and 1 value signal. The global aggregator has its own residual tower (N_G blocks, hidden dimension H_G). The fusion input dimension is $H_G + |\mathcal{A}| + P$, where P is the number of patches.

Transfer training protocol.

1. **Pre-train** a standard AlphaZeroBitNet on the $k \times k$ board until convergence.
2. **Freeze** the local model: all parameters have `requires_grad=false` and the model runs in evaluation mode (no dropout).
3. **Train only the global aggregator** on the large board via normal self-play. Since local tactics are pre-learned, the bootstrap phase can be skipped entirely.

Advantages.

- ∼60% fewer trainable parameters than a monolithic network of comparable capacity.
- Faster convergence: local tactical patterns do not need re-discovery.
- Natural spatial decomposition without weight-sharing assumptions of CNNs.

This approach is analogous to CNN local receptive fields but operates on the graph structure of the Dots-and-Boxes grid rather than a regular pixel lattice.

4.4 Monte Carlo Tree Search

We implement AlphaZero-style MCTS with the PUCT selection rule [9]:

$$a^* = \arg\max_a \left(Q(s,a) + c_{\text{puct}}\, P(s,a)\, \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)} \right), \tag{4}$$

where $P(s,a)$ is the policy prior from the network, $N(s,a)$ are visit counts, and $Q(s,a) = W(s,a)/N(s,a)$ is the mean action value.

Dirichlet exploration noise. At the root node during self-play, priors are perturbed:

$$P'(s,a) = (1-\varepsilon)\, P(s,a) + \varepsilon\, \eta_a, \quad \eta \sim \text{Dir}(\alpha). \tag{5}$$

Algorithm 1 Asynchronous MCTS root search (conceptual)

```
for sim = 1 to S do
  n ← root
  while n is expanded and not terminal do
    n ← SelectChild(n) via PUCT
    if n.status = PENDING then
      continue to next simulation
    end if
  end while
  if n is terminal then
    Backpropagate(path, outcome)
  else
    n.status ← PENDING
    EnqueueInference(n.state)
  end if
end for
On inference reply (n, p, v):
  Expand(n, p); Backpropagate(path, v)
```

First-play urgency (FPU) reduction. Unvisited actions ($N(s,a) = 0$) use a pessimistic value estimate:

$$Q_{\text{FPU}}(s,a) = \bar{Q}(s) - \delta_{\text{fpu}}, \tag{6}$$

where $\bar{Q}(s)$ is the parent's mean value and $\delta_{\text{fpu}} \in [0,1]$ is the FPU reduction parameter. This prevents premature uniform exploration when the policy prior is uncalibrated early in training.

Extra-turn value conversion. Dots-and-Boxes includes "extra move" captures where the current player retains the turn. During PUCT selection, if a child node has the *same* player as the parent (an extra-turn edge), $Q(s,a)$ is used directly; if the player changes, $Q(s,a)$ is *negated* to convert to the parent's perspective. This is critical for correctly valuing capture sequences.

Temperature-based action selection. The action played in self-play is sampled proportionally to visit counts raised to a temperature:

$$\pi(a) \propto N(s,a)^{1/\tau}, \qquad \tau = \begin{cases} \tau_{\text{explore}} & \text{if move count} < t_{\text{thresh}}, \\ \tau_{\text{exploit}} & \text{otherwise.} \end{cases} \tag{7}$$

This schedule encourages diverse openings (high τ) while playing near-greedily ($\tau \to 0$) in the mid- and endgame.

4.5 Asynchronous MCTS

In synchronous MCTS, each simulation blocks at every leaf until the neural network returns $(\mathbf{p}, v)$. When inference is the bottleneck, this underutilises CPU cores. We implement an asynchronous variant using *pending nodes*:

When a simulation reaches an unexpanded leaf, it submits an inference request, marks the node PENDING, and immediately starts a new simulation. If a simulation encounters a pending node, it skips that path. When the inference result arrives, the node transitions to EXPANDED with

initialised priors. This overlaps CPU-side selection/backup with inference and increases simulations per second.

4.6 Tree vs. DAG Search

The implementation supports both tree-based search and a hybrid DAG mode controlled by a configuration flag:

- **Training (self-play):** Tree mode (default). Each simulation path allocates fresh nodes, avoiding shared-statistics effects during learning while policies and values are unstable.
- **Evaluation:** DAG mode. A per-move transposition table maps a 128-bit hash of the full board state to shared node pointers. Identical states reached via different move orders reuse the same node, reducing duplicated expansions. The hash uses golden-ratio mixing of four bitset hash pairs plus the current player, providing collision probability $< 2^{-64}$ for boards up to 20×20. The table is cleared after each move to bound memory.

Additionally, an *inference cache* avoids redundant neural network evaluations within a single `act()` call: if the same board state is reached by multiple simulation paths, the cached policy–value result is reused without re-querying the inference server.

4.7 Capture Boost Prior

During the early bootstrap phase when the network's policy is essentially random, MCTS may fail to discover that completing a box is always beneficial. To address this cold-start problem, we optionally apply a *capture boost*: the network's prior probability for actions that complete at least one box is multiplicatively boosted by a factor β_{cap} before normalisation. We linearly decay β_{cap} to zero across training phases as the network learns to prioritise captures on its own.

4.8 Training Pipeline

4.8.1 Training Targets and Loss

For each state s_t visited during self-play, we store a tuple $(x_t, \pi_t, \mathbf{m}_t, z)$ where x_t is the encoded state, π_t is the policy target from MCTS visit counts, $\mathbf{m}_t$ is the legal-action mask, and z is the value target.

The network output is $(\hat{\mathbf{p}}, \hat{v}) = f_\theta(x)$. We minimise the loss:

$$\mathcal{L}(\theta) = \underbrace{-\sum_a \pi(a) \log \hat{p}(a)}_{\mathcal{L}_\pi} + 1.5 \underbrace{(\hat{v} - z)^2}_{\mathcal{L}_v} \tag{8}$$

where the policy loss uses masked log-softmax ($\hat{p}(a) = \text{softmax}(\mathbf{l} + (\mathbf{m} - 1) \cdot 10^9)_a$). The value loss coefficient of 1.5 (rather than the standard 1.0 used in AlphaZero [12]) was found empirically to stabilise early training by providing a stronger gradient signal for value estimation.

Optimisation. We use Adam [6] with per-phase learning rates. Gradient norms are clipped to $\|\nabla\|_2 \leq 1.0$ to prevent instability from rare outlier states. The number of gradient steps per epoch is capped at $\min(|\text{buffer}|/B_{\text{batch}}, 256)$ to ensure stable updates even as the replay buffer grows.

Table 2: Value target definitions. $B = rc$ is the total number of boxes, $\Delta = s_{\text{me}} - s_{\text{opp}}$.

Name	Formula	Range
WinLoss	$z = \text{sign}(\Delta)$	$\{-1, 0, +1\}$
ScoreDiff	$z = \Delta/B$	$[-1, +1]$
ScoreDiffSqrt	$z = \text{sign}(\Delta)\sqrt{\lvert\Delta\rvert/B}$	$[-1, +1]$
ScoreDiffTanh	$z = \tanh(2\Delta/B)$	$(-1, +1)$
ScoreDiffScaled	$z = \tanh(0.5 \cdot \Delta)$	$(-1, +1)$

Table 3: Phased training curriculum (example for 5×5 board). S: MCTS sims, η: learning rate, τ_e/τ_x: explore/exploit temperature, β: capture boost.

Phase	Iters	S	η	τ_e/τ_x	β	Purpose
Bootstrap	30	250	0.003	1.0/0.3	0.3	Cold-start; learn captures
ChainAware	80	600	0.001	0.9/0.15	0.0	Chain reasoning
DeepSearch	70	1000	0.0003	0.8/0.08	0.0	Endgame/parity
Mastery	50	1400	0.0001	0.6/0.02	0.0	Near-greedy exploitation

4.8.2 Value Target Design

Dots-and-Boxes yields final scores s_1, s_2 with margin $\Delta = s_{\text{me}} - s_{\text{opp}}$. We implement five value targets, offering a spectrum from sparse to dense supervision:

The signal-crushing problem. The ScoreDiffTanh target divides by B before applying tanh. On large boards ($B = 25$ for 5×5), a 1-box lead yields $z = \tanh(2/25) = \tanh(0.08) \approx 0.08$, providing minimal gradient signal. Values cluster in $[-0.2, +0.2]$, making it difficult for the network to distinguish winning from losing positions.

Board-size-invariant target. ScoreDiffScaled addresses this by applying tanh to the *raw* (unnormalized) score difference: $z = \tanh(0.5 \cdot \Delta)$. This gives:

$$\begin{array}{cccc} \Delta = 1 & \Delta = 2 & \Delta = 3 & \Delta = 5 \\ z \approx 0.46 & z \approx 0.76 & z \approx 0.91 & z \approx 0.99 \end{array}$$

The mapping is independent of board size: a 1-box lead always yields $z \approx 0.46$ whether on a 3×3 or 7×8 board, providing a strong, discriminative training signal.

4.8.3 Phased Training Curriculum

Rather than using fixed hyperparameters throughout training, we employ a multi-phase curriculum that adapts key parameters as the agent improves. Each phase specifies its own MCTS simulation budget, learning rate, temperature schedule, and capture boost:

The **Bootstrap** phase uses lower simulation budgets and higher exploration to rapidly accumulate diverse experience. The optional capture boost ensures MCTS always explores capture moves during the cold-start period. **ChainAware** increases MCTS depth so the agent can discover multi-step chain strategies. **DeepSearch** further increases simulations to refine endgame and parity play. **Mastery** uses near-greedy exploitation with very low learning rate for final polishing.

For PatchNet, the Bootstrap phase can be skipped entirely since local tactics are pre-learned from the frozen small-board model.

4.8.4 Self-Play, Replay Buffer, and Batched Inference

Self-play workers. Training is parallelised across W worker threads. Each worker runs complete self-play games using MCTS and pushes training tuples to a shared replay buffer. Workers communicate with the inference server via asynchronous submit/poll: the worker submits a state snapshot and polls for the response without blocking.

Batched inference server. A dedicated inference thread aggregates requests from all workers into batches of up to $B_{\max}$ states. After a brief accumulation window ($\sim$200 μs) or when the batch reaches capacity, the server runs a single forward pass, applies softmax to obtain policy probabilities, and dispatches results back to workers. This amortises GPU/CPU kernel launch overhead and improves hardware utilisation.

Replay buffer. The replay buffer is a capacity-limited circular queue implemented as a thread-safe `std::deque`. When the buffer is full, the oldest samples are discarded (FIFO eviction). The buffer capacity grows adaptively across iterations (e.g., +2000 per iteration) to accommodate the expanding diversity of self-play experience. Random minibatches are drawn using a partial Fisher–Yates shuffle for $O(\text{batch_size})$ sampling complexity.

For reproducibility and training resumption, the buffer supports serialisation: state, policy, mask tensors and value scalars are stacked and written to a LibTorch archive. Optimizer state and phase/iteration progress are checkpointed alongside the model weights.

4.8.5 Symmetry Augmentation

Dots-and-Boxes boards admit geometric symmetries that preserve the game dynamics. We exploit these for zero-cost data augmentation by remapping state features and policy targets under symmetry transformations.

- **Rectangular boards** ($r \neq c$): horizontal flip and vertical flip (2 transforms $\Rightarrow$ 3$\times$ data with identity).
- **Square boards** ($r = c$): the full dihedral group D_4 comprising identity, 2 reflections, 3 rotations, and 2 diagonal flips (7 transforms $\Rightarrow$ 8$\times$ data).

Unlike image augmentation where pixel indices transform uniformly, Dots-and-Boxes requires separate handling of horizontal and vertical edge sets: under rotation, horizontal edges map to vertical edges and vice versa. The augmentation module pre-computes index permutation tables for each transform, then generates augmented training samples by bitset remapping—a pure $O(|E|)$ operation with no neural network calls.

Currently, symmetry augmentation is disabled for PatchNet because patch features would require separate permutation tables; extending augmentation to PatchNet is left for future work.

5 Implementation Details

Language and framework. The system is written in C++17 and uses the PyTorch C++ frontend (LibTorch) for neural network definition, training, and inference. CUDA is used automatically when available; the system degrades gracefully to CPU-only execution.

Table 4: Default network and MCTS hyperparameters by board size.

Parameter	2×2	3×3	5×5	5×5 (Patch)
Hidden size H	64	128	384	256 (global)
Residual blocks N_B	4	6	10	6 (global)
Dropout p	0.1	0.1	0.06	0.08
MCTS sims S	200	400–800	600–1400	400–1200
c_{puct}	1.5	1.6	1.6	1.6
Dirichlet α	0.3	0.15	0.10	0.10
Dirichlet ε	0.25	0.20	0.25	0.25
FPU reduction	0.25	0.20	0.20	0.20
Workers W	8	16	16	16
Batch size	128	256	256	256
Buffer capacity	20K	50K	60K	50K
Value target	WL	SD Tanh	SD Scaled	SD Scaled
Augmentation	D_4	D_4	off	off

Hyperparameters. Table 4 lists the default hyperparameters for each board size.

Reproducibility. The full source code, including all configuration files and evaluation scripts, is publicly available at https://github.com/Nishant040305/Dot-Boxes.

6 Experimental Setup

6.1 Baselines

We evaluate against the following opponents:

- **Random**: uniform random legal move selection.
- **Greedy**: always captures a free box (count-3) if available; otherwise plays randomly. This is the minimal tactical baseline.
- **Alpha–Beta**: depth-limited minimax with alpha–beta pruning, transposition tables, and move ordering. Depths tested: $d \in \{2, 3, 4, 6\}$ where computationally feasible.

6.2 Evaluation Protocol

For each agent-vs-baseline pair, we play $n \geq 100$ evaluation games with alternating first player. The agent uses greedy action selection (temperature $\tau \to 0$) during evaluation. We report:

- **Win-rate** (% of games won by the trained agent).
- **Average score difference** ($\bar{\Delta}$).
- **Compute budget**: MCTS simulations per move and wall-clock time per move.

6.3 Board Sizes

- 2×2: 12 edges. Rapid iteration and debugging.
- 3×3: 24 edges. Standard benchmark where chains and tactical patterns emerge.
- 5×5: 60 edges. Tests scalability; PatchNet vs. monolithic comparison.

7 Results

7.1 Win-Rates and the Effect of Simulation Budget

We evaluate the agent at varying MCTS simulations per move to understand how planning scales with computation. Performance against stronger opponents (Greedy, Alpha–Beta) generally improves with increased simulations, indicating that the agent effectively leverages deeper search to outmaneuver tactical play. On the 3×3 board, the agent achieves a 100.0% win-rate against Greedy with 400 simulations per move and maintains this performance at higher budgets. Against a depth-3 Alpha–Beta opponent, the agent wins 95.0% of games with 800 simulations, improving further to 99.0% at 3200 simulations.

Non-monotonic scaling on 2×2. A notable exception to the above trend is observed on the 2×2 board, where win-rates against weaker opponents (Random, Greedy) exhibit a *non-monotonic, approximately parabolic relationship* with the simulation budget. Specifically, increasing simulations from 200 to 800 leads to a decrease in win-rate, followed by a recovery at higher budgets (e.g., 3200 simulations).

GTO vs. exploitative behavior. This phenomenon arises from the distinction between *game-theoretic optimal (GTO)* play and *exploitative* play. With larger simulation budgets, MCTS increasingly evaluates positions under the assumption of a perfect opponent and therefore prioritizes *risk-averse, worst-case optimal strategies.* On the 2×2 board—where many lines provably lead to draws—the agent prefers moves that guarantee a draw over moves that could yield a win only if the opponent errs. As a result, the agent effectively *trades off exploitability for optimality*, leading to increased draw rates and reduced empirical win-rate against suboptimal opponents.

Effect of low simulation budgets. At lower simulation counts, the agent relies more heavily on its learned policy prior, resulting in more diverse and less constrained play. This induces more complex and unstable board states, which increases the likelihood of mistakes from weaker opponents. Consequently, the agent achieves higher win-rates despite being theoretically less optimal.

Second-mover advantage in 2×2. Additionally, the 2×2 environment exhibits a strong *second-mover advantage.* Due to the small state space, optimal play from the second player often yields a forced win or decisive advantage, whereas the first player is largely restricted to defensive play that frequently results in draws. This structural asymmetry further amplifies sensitivity to simulation depth and contributes to the observed non-monotonic (parabolic) scaling behavior.

Table 5: Comprehensive percentage of Wins (W), Losses (L), and Draws (D) of the trained AlphaZero agent against baselines at varying MCTS simulation budgets.

Board	Opponent	Sims/move	W (%)	L (%)	D (%)	$\bar{\Delta}$	Time/move
2×2	Random	200	87.5	0.0	12.5	+2.90	88.6 ms
2×2	Random	400	85.2	0.0	14.8	+2.79	140.7 ms
2×2	Random	800	83.8	0.0	16.2	+2.88	162.5 ms
2×2	Random	1600	84.2	0.2	15.5	+2.86	275.0 ms
2×2	Random	3200	91.8	0.0	8.2	+2.99	323.0 ms
2×2	Greedy	200	54.8	2.8	42.5	+1.79	87.4 ms
2×2	Greedy	400	52.8	2.2	45.0	+1.83	143.3 ms
2×2	Greedy	800	58.5	1.2	40.2	+1.88	205.9 ms
2×2	Greedy	1600	63.2	1.0	35.8	+2.00	241.7 ms
2×2	Greedy	3200	76.0	3.8	20.2	+2.28	331.2 ms
2×2	Alpha–Beta (d=3)	200	50.0	15.0	35.0	+1.21	73.5 ms
2×2	Alpha–Beta (d=3)	400	50.0	12.8	37.2	+1.29	110.2 ms
2×2	Alpha–Beta (d=3)	800	51.2	12.2	36.5	+1.39	104.3 ms
2×2	Alpha–Beta (d=3)	1600	50.0	12.0	38.0	+1.33	233.5 ms
2×2	Alpha–Beta (d=3)	3200	70.8	5.2	24.0	+2.31	257.4 ms
3×3	Random	200	100.0	0.0	0.0	+8.08	282.9 ms
3×3	Random	400	100.0	0.0	0.0	+8.11	472.1 ms
3×3	Random	800	100.0	0.0	0.0	+8.12	679.4 ms
3×3	Random	1600	100.0	0.0	0.0	+8.24	1040.3 ms
3×3	Random	3200	100.0	0.0	0.0	+8.33	2110.8 ms
3×3	Greedy	200	100.0	0.0	0.0	+5.41	273.4 ms
3×3	Greedy	400	100.0	0.0	0.0	+5.46	430.0 ms
3×3	Greedy	800	100.0	0.0	0.0	+5.43	748.0 ms
3×3	Greedy	1600	100.0	0.0	0.0	+5.52	1545.0 ms
3×3	Greedy	3200	100.0	0.0	0.0	+5.83	2880.0 ms
3×3	Alpha–Beta (d=3)	200	94.0	6.0	0.0	+3.40	358.8 ms
3×3	Alpha–Beta (d=3)	400	96.0	4.0	0.0	+3.34	367.2 ms
3×3	Alpha–Beta (d=3)	800	95.0	5.0	0.0	+3.42	584.1 ms
3×3	Alpha–Beta (d=3)	1600	100.0	0.0	0.0	+3.90	946.5 ms
3×3	Alpha–Beta (d=3)	3200	99.0	1.0	0.0	+3.68	1810.2 ms

Note: 5×5 board experiments are ongoing and will be reported in an extended version of this work.

7.2 Depth-Based Analysis Against Alpha–Beta

We further challenge the agent against the Alpha–Beta search with varying analytical depths. For this comparison, the AlphaZero agent plays with a fixed 3200 simulations per move.

Table 7: Compute comparison between alpha–beta and AlphaZero MCTS. Alpha–beta times are from sequential search on an Intel i7-13700H.

Method	Board	Depth/Sims	Time/move
Minimax	5×5	d=3	∼2 min
Alpha–Beta	5×5	d=3	∼30 s
Alpha–Beta	5×5	d=4	∼1 min
Alpha–Beta	5×5	d=6	∼7 min

Table 6: AlphaZero (3200 sims/move) vs Alpha–Beta search at increasing depths shown as normalized percentages.

Board	AB Depth	W (%)	L (%)	D (%)	$\bar{\Delta}$	Time/move
2×2	d=4	62.0	12.0	26.0	+1.80	330.4 ms
2×2	d=5	58.0	22.0	20.0	+0.72	287.2 ms
2×2	d=6	50.0	28.0	22.0	+0.44	282.2 ms
2×2	d=7	50.0	22.0	28.0	+0.56	293.1 ms
2×2	d=8	50.0	36.0	14.0	+0.28	312.7 ms
2×2	d=9	50.0	50.0	0.0	+0.00	324.5 ms
2×2	d=10	50.0	50.0	0.0	+0.00	307.3 ms
3×3	d=3	99.0	1.0	0.0	+3.68	1810.2 ms
3×3	d=4	96.0	4.0	0.0	+3.42	1852.6 ms
3×3	d=5	94.0	6.0	0.0	+3.28	1984.7 ms
3×3	d=6	92.0	8.0	0.0	+2.84	2568.7 ms
3×3	d=7	92.0	8.0	0.0	+2.76	2570.2 ms

Despite playing against deep alpha–beta search capable of seeing up to $d = 10$ plies ahead on the 2×2 board and $d = 7$ plies ahead on the 3×3 board, the AlphaZero agent maintains a strong advantage. On the 3×3 board, it achieves over a 90% win rate even against d=7. On the 2×2 board, it maintains at least a 50% win rate against exhaustive search (up to d=10).

7.3 Scalability: AlphaZero MCTS vs. Alpha–Beta

Traditional search methods exhibit exponential time growth with depth. Alpha–beta pruning and move ordering improve feasibility (depth-6 reduced from infeasible minimax to ∼7 minutes), but remain impractical for real-time play on boards larger than 5×5. In contrast, the AlphaZero agent amortises planning computation into the neural network, maintaining bounded per-move latency (typically under 2 seconds on the 3×3 board even at 3200 simulations) regardless of the underlying state-space complexity.

7.4 PatchNet vs. Monolithic

The PatchNet architecture is designed to reduce the computational burden of training on larger boards by reusing locally learned features. On the 5×5 board, PatchNet employs a frozen 3×3 pre-trained model to extract features from 9 overlapping patches, training only a lightweight global aggregator (256 hidden units, 6 residual blocks) compared to the monolithic network (384 hidden

units, 10 residual blocks). This yields an estimated ∼60% reduction in trainable parameters. Full comparative evaluation on the 5×5 board—including convergence speed, final win-rates, and parameter counts—is ongoing and will be reported in a future extended version.

7.5 Training Dynamics

Across all board sizes, we observe a consistent training progression. During the Bootstrap phase, the policy loss decreases rapidly as the agent learns basic capture tactics. The transition to the ChainAware phase—marked by increased MCTS simulations and reduced exploration—correlates with the emergence of multi-step chain reasoning, visible as a temporary increase in value loss followed by convergence. On the 3×3 board, the agent achieves 100% win-rate against the Greedy baseline within the first two training phases, confirming that local tactical mastery is acquired early. Subsequent phases refine positional judgement and endgame play, leading to strong performance against deeper alpha–beta search.

8 Ablation Studies

Ablation experiments are designed to isolate the contribution of individual components. The following table outlines the planned comparisons; full results will be included upon completion of the experimental campaign.

Table 8: Ablation results on the 3×3 board. Each row modifies one component from the full configuration. Win-rate is measured against Greedy over 200 games.

Configuration	Win-rate (%)	Δ from full
Full system (baseline)	–	–
No Dirichlet noise ($\varepsilon = 0$)	–	–
No edge-count one-hot features	–	–
WinLoss value target (instead of ScoreDiffTanh)	–	–
No symmetry augmentation	–	–
FPU reduction = 0	–	–
Half MCTS simulations (200 instead of 400)	–	–
Double MCTS simulations (800)	–	–

9 Analysis and Discussion

Dominance over heuristic baselines. On the 3×3 board, the agent achieves a perfect 100% win-rate against both Random and Greedy baselines across all tested simulation budgets (200–3200), with average score differences of +8.3 and +5.8 respectively. This indicates that the agent has fully internalised both capture tactics and deeper strategic reasoning (e.g., chain management), rendering heuristic opponents non-competitive.

Scaling of search budget. Increasing the MCTS simulation budget generally improves performance against stronger opponents. On the 3×3 board, win-rate against depth-3 alpha–beta search improves from 94% at 200 simulations to 100% at 1600 simulations. However, on the 2×2 board, we observe a non-monotonic relationship: higher simulation counts initially reduce win-rate against weak opponents before recovering at very high budgets. We attribute this to the agent converging toward game-theoretically optimal (GTO) play—preferring guaranteed draws over risky wins—when given sufficient planning depth in the small, highly constrained state space.

Robustness against deep search. Even against alpha–beta search with increasing depth, the agent remains competitive. On the 3×3 board, it maintains a 92% win-rate against d=7 alpha–beta with 3200 simulations per move. On the 2×2 board, the agent achieves 50% win-rate against d=9 and d=10 (exhaustive search), reflecting convergence to optimal play where wins are evenly split by starting position. The graceful degradation pattern (from 70.8% at d=3 to 50% at d=10) suggests the agent approaches near-optimal play.

Computational efficiency. The AlphaZero agent computes moves in hundreds of milliseconds to low single-digit seconds, whereas alpha–beta search on the 5×5 board requires 30 seconds to 7 minutes per move even at moderate depths. This orders-of-magnitude improvement in per-move latency demonstrates the practical advantage of amortising strategic knowledge into a neural network.

GTO–exploitative trade-off. The non-monotonic scaling pattern on the 2×2 board highlights a fundamental tension in self-play systems: agents trained with deeper search converge toward minimax-optimal strategies that may underperform against suboptimal opponents. This finding

has implications for deployment, where opponents are rarely optimal, and suggests that adaptive simulation budgets or opponent-modelling may be beneficial.

10 Limitations and Future Work

Current limitations.

- Dots-and-Boxes has known strategic motifs (chains, parity, nimstring values [4]) that the fully-connected network must learn from scratch. Explicit chain-detection features or graph neural networks could provide useful inductive bias.
- PatchNet currently does not support symmetry augmentation because patch features require separate permutation tables.
- The fully-connected architecture does not exploit the spatial locality of the board as efficiently as convolutional or graph-based architectures might.
- Training was conducted only on CPU; GPU acceleration would enable scaling to larger boards and higher simulation budgets.

Future directions.

- **Convolutional and graph-based architectures**: replacing the FC backbone with a GNN or CNN that respects the grid topology.
- **Curriculum learning across board sizes**: progressively increasing board size during training, using PatchNet-style transfer.
- **Learned dynamics model** (MuZero-style [11]): eliminating the need for an explicit game simulator during planning.
- **Evaluation against human experts** and optimal solvers on small boards [3].
- **Multi-agent population training** to avoid strategy collapse from pure self-play.

11 Conclusion

We presented a self-play reinforcement learning system for Dots-and-Boxes that extends the AlphaZero paradigm with three methodological contributions: the PatchNet hierarchical architecture for spatial transfer of locally learned tactics, a four-phase training curriculum that co-adapts search depth and exploration across training, and a board-size-invariant value target that avoids gradient signal crushing on larger boards.

On the 3×3 board, the trained agent achieves a 100% win-rate against greedy baselines and up to 99% against depth-3 alpha–beta search, while maintaining 92% even against depth-7 search. The agent computes moves in under 2 seconds compared to minutes required by traditional alpha–beta at comparable strategic depth. On the 2×2 board, analysis of the non-monotonic scaling behaviour reveals a trade-off between game-theoretically optimal and exploitative play, providing insight into the interaction between search depth and opponent strength.

These results demonstrate that AlphaZero-style learning is effective for combinatorial games with delayed rewards and extra-turn mechanics, and that hierarchical spatial transfer offers a promising direction for scaling learned game-playing agents to larger and more complex domains.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Trapit Bansal, Jakub Pachocki, Szymon Sidor, Ilya Sutskever, and Igor Mordatch. Emergent complexity via multi-agent competition. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.

[3] Joseph K. Barker and Richard E. Korf. Solving Dots-and-Boxes. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 414–419, 2012.

[4] Elwyn R. Berlekamp. *The Dots and Boxes Game: Sophisticated Child's Play.* A K Peters/CRC Press, 2000.

[5] Rémi Coulom. Efficient selectivity and backup operators in Monte-Carlo tree search. In *International Conference on Computers and Games*, pages 72–83. Springer, 2006.

[6] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015.

[7] Levente Kocsis and Csaba Szepesvári. Bandit based Monte-Carlo planning. In *European Conference on Machine Learning*, pages 282–293. Springer, 2006.

[8] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992.

[9] Christopher D. Rosin. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230, 2011.

[10] Jonathan Schaeffer, Neil Burch, Yngvi Björnsson, Akihiro Kishimoto, Martin Müller, Robert Lake, Paul Lu, and Steve Sutphen. Checkers is solved. *Science*, 317(5844):1518–1522, 2007.

[11] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.

[12] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

[13] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.